Database Design Guide

This guide will help the owner to create a database on the Inventory Management System. It will help to manage the below functionalities.

- Supplier Details
- Product Details
- Customer Details
- Order Details
- Delivery Details
- Defective Products

We will use MySQL as the DBMS to create the database and its related operations.

1. Introduction to MySQL:

MySQL is an open-source relational database management system (RDBMS) that uses structured query language (SQL) to manage and manipulate data in a database. It is widely used for various applications, from small web applications to large enterprise systems.

MySQL's key features include:

- Scalability: Capable of handling large amounts of data and concurrent connections.
- Flexibility: Supports various data types and storage engines.
- Performance: Optimized for speed and efficiency.
- Reliability: Known for its stability and robustness.

2. Installation of MySQL:

MySQL can be installed on various operating systems, including Windows, macOS, and Linux. Here are the general steps to install MySQL:

Windows:

- Download the MySQL installer from the official website. https://dev.mysql.com/downloads/installer/
- Run the installer and follow the on-screen instructions.
- Choose the installation type (Typical, Complete, or Custom). Recommended Custom.
- Set a root password for the MySQL server.

3. E-R Diagram (ERD):

An Entity-Relationship Diagram (ERD) is a visual representation of the data model that shows the entities, attributes, relationships between entities, and cardinality. ERDs are commonly used in database design to help developers and stakeholders understand the structure and relationships within a database.

i) Identify Entities:

- Start by identifying the main entities in your system. These are the objects or concepts about which you want to store data.
- Each entity should correspond to a table in your database.

ii) Define Attributes:

- For each entity, list the attributes (properties or fields) that describe it.
- These attributes will become columns in the corresponding database table.

iii) Identify Relationships:

- Determine how entities are related to each other. There are three types of relationships: one-to-one (1:1), one-to-many (1:M), and many-to-many (M:M).
- Represent these relationships using lines connecting the entities.
- Let's see a few examples of relationships:

One to One

One to Many

Many to One

Many to Many

iv) Optional:

a) **Add Attributes and Constraints:**
- Include additional information in your ERD, such as primary keys, foreign keys, and constraints (e.g., unique constraints).

b) **Create the Diagram**:
- Use specialized diagramming software or tools (e.g., Lucidchart, draw.io, or even pen and paper) to create your ERD.

c) **Refine and Review:**
- Review your ERD with stakeholders and team members to ensure it accurately represents the data model and relationships. Make any necessary refinements.

4. Let's identify the attributes and relationships of each entities for the Inventory Management System:

1) <u>Supplier:</u>

- **Attributes:**
  Supplier_Id (Primary Key)
  Name
  Address
  Mobile Number

- **Relationships:**
  One **Supplier** can have multiple **products (One-to-Many)**

2) <u>Products:</u>

- **Attributes**:
  Product_Id (Primary Key)
  Product Name
  Description
  Category
  Product_Quantity
  Supplier_Name (Foreign key)

- **Relationships:**
  Many **Products** can be in single **category** (**Many-to-One**)
  Many **Products** can have single **supplier (One-to-Many)**

3) <u>Customer:</u>

- **Attributes:**
  Customer_Id (Primary Key)
  Name
  Address
  Mobile Number

- **Relationships:**
  One **Customer** can buy multiple **products** (**One-to-Many**)
  One **Customer** can buy single **product** also **(One-to-One)**

4) <u>Order:</u>

- **Attributes:**
  Order_Id (Primary Key)
  Product Name (Foreign key)
  Order_Quantity
  Customer_Name (Foreign key)
  Customer_Address (Foreign Key)
  Customer_MobileNumber (Foreign key)
  Order_Date

- **Relationships:**
  One **Order** can placed by multiple **customer** (**One-to-Many**)
  Many **Order** can placed through single **customer** (**Many-to-One**)

5) **Delivery:**

- **Attributes**:
  Delivery_Id (Primary Key)
  Product Name (Foreign key)
  Delivery_Quantity
  Order_Date (Foreign Key)
  Delivery_Date

- **Relationships:**
  Many **Deliveries** will done by one **Person (Many-to-one)**

6) **Defective:**

- **Attributes**:
  Product Name (Foreign key)
  Defective_Quantity
  Supplier_Name (Foreign key)
  Supplier_MobileNumber(Foreign key)

- **Relationships**:
  Many **Defective** quantity will be there of single **Product** (**One-to-Many**)

5. Table Structure:

1) Supplier Details:

```
mysql> desc supplier_details;
+-----------------------+--------------+------+-----+---------+-------+
| Field                 | Type         | Null | Key | Default | Extra |
+-----------------------+--------------+------+-----+---------+-------+
| Supplier_Id           | int          | NO   | PRI | NULL    |       |
| Supplier_Name         | varchar(100) | YES  | UNI | NULL    |       |
| Supplier_Address      | varchar(200) | YES  |     | NULL    |       |
| Supplier_MobileNumber | bigint       | YES  | UNI | NULL    |       |
+-----------------------+--------------+------+-----+---------+-------+
4 rows in set (0.03 sec)
```

2) Product Details:

```
mysql> desc product_details;
+---------------------+--------------+------+-----+---------+-------+
| Field               | Type         | Null | Key | Default | Extra |
+---------------------+--------------+------+-----+---------+-------+
| Product_Id          | int          | NO   | PRI | NULL    |       |
| Product_Name        | varchar(100) | YES  | UNI | NULL    |       |
| Product_Description | varchar(200) | YES  |     | NULL    |       |
| Product_Category    | varchar(100) | YES  |     | NULL    |       |
| Product_Quantity    | bigint       | YES  |     | NULL    |       |
| Supplier_Name       | varchar(100) | YES  | MUL | NULL    |       |
+---------------------+--------------+------+-----+---------+-------+
6 rows in set (0.00 sec)
```

3) Customer Details:

```
mysql> desc customer_details;
+-----------------------+--------------+------+-----+---------+-------+
| Field                 | Type         | Null | Key | Default | Extra |
+-----------------------+--------------+------+-----+---------+-------+
| Customer_Id           | int          | NO   | PRI | NULL    |       |
| Customer_Name         | varchar(100) | YES  | UNI | NULL    |       |
| Customer_Address      | varchar(200) | YES  | UNI | NULL    |       |
| Customer_MobileNumber | bigint       | YES  | UNI | NULL    |       |
+-----------------------+--------------+------+-----+---------+-------+
4 rows in set (0.00 sec)
```

4) Order Details:

```
mysql> desc order_details;
+-----------------------+--------------+------+-----+---------+-------+
| Field                 | Type         | Null | Key | Default | Extra |
+-----------------------+--------------+------+-----+---------+-------+
| Order_Id              | int          | NO   | PRI | NULL    |       |
| Order_Quantity        | bigint       | YES  |     | NULL    |       |
| Customer_Name         | varchar(100) | YES  | MUL | NULL    |       |
| Customer_Address      | varchar(200) | YES  | MUL | NULL    |       |
| Customer_MobileNumber | bigint       | YES  | MUL | NULL    |       |
| Order_Date            | date         | YES  | UNI | NULL    |       |
| Product_Name          | varchar(100) | YES  | MUL | NULL    |       |
+-----------------------+--------------+------+-----+---------+-------+
7 rows in set (0.01 sec)
```

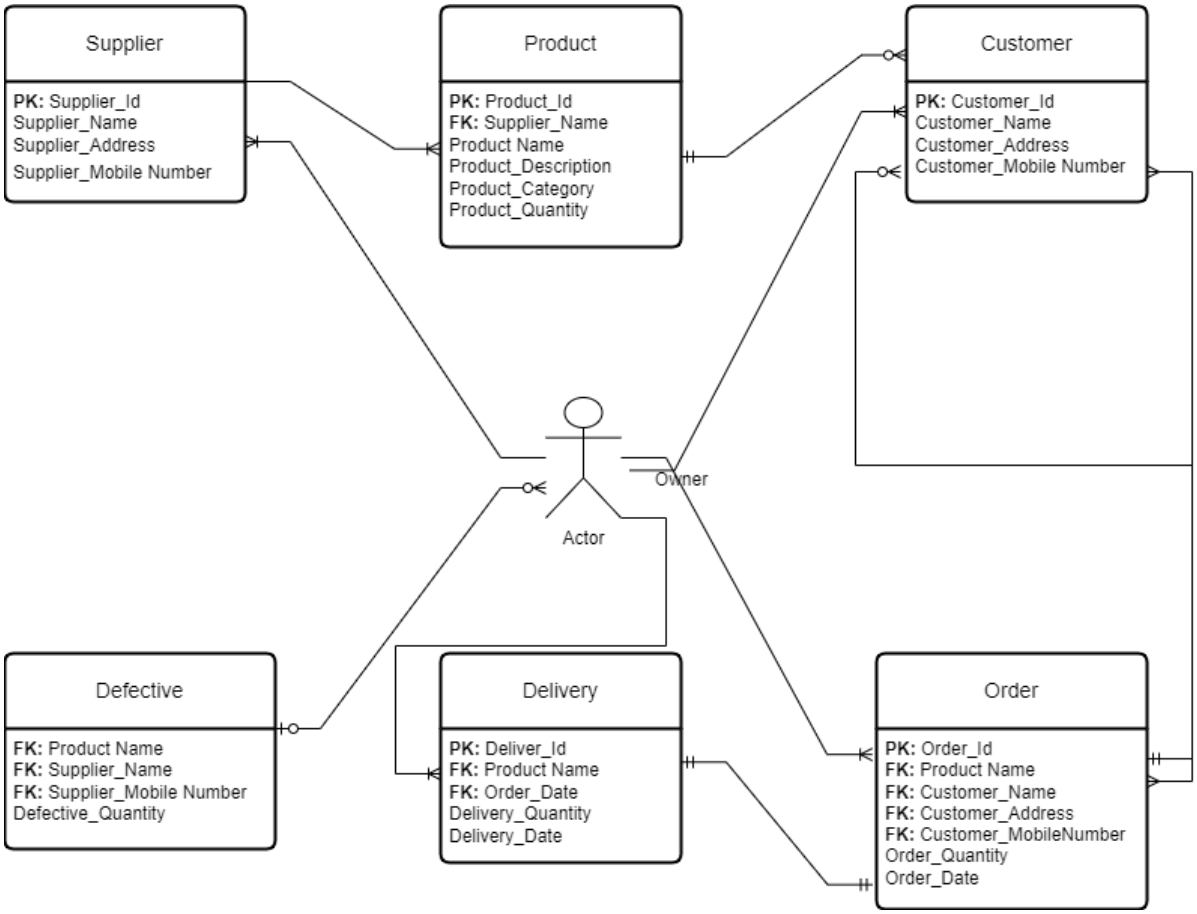
5) Delivery Details:

```
mysql> desc delivery_details;
+--------------------+--------------+------+-----+---------+-------+
| Field              | Type         | Null | Key | Default | Extra |
+--------------------+--------------+------+-----+---------+-------+
| Delivery_Id        | int          | YES  |     | NULL    |       |
| Delivery_Quantity  | bigint       | YES  |     | NULL    |       |
| Delivery_Date      | date         | YES  |     | NULL    |       |
| Product_Name       | varchar(100) | YES  | MUL | NULL    |       |
| Order_Date         | date         | YES  | MUL | NULL    |       |
+--------------------+--------------+------+-----+---------+-------+
5 rows in set (0.00 sec)
```

6) Defective Products:

```
mysql> desc defective_products;
+-----------------------+--------------+------+-----+---------+-------+
| Field                 | Type         | Null | Key | Default | Extra |
+-----------------------+--------------+------+-----+---------+-------+
| Product_Name          | varchar(100) | YES  | MUL | NULL    |       |
| Defective_Quantity    | bigint       | YES  |     | NULL    |       |
| Supplier_Name         | varchar(100) | YES  | MUL | NULL    |       |
| Supplier_MobileNumber | varchar(200) | YES  |     | NULL    |       |
+-----------------------+--------------+------+-----+---------+-------+
4 rows in set (0.00 sec)
```

6. **Let's create the ER diagram to visually represent the entities and relationships:**



- **In this ERD:**
  i. Owner have multiple supplier and multiple customer, creating a many-to-many relationship.
  ii. Multiple products deliver by on supplier (many-to-one relationship).
  iii. Each Customer buy multiple products (one-to-many relationship).
  iv. Customer can order one or more than one product.
  v. One or Many products may be defective.

7. **Creating a Database:**
   Using MySQL server, create a new database for your Inventory management system. You can do this with SQL commands or through the graphical interface.

   CREATE DATABASE IMS;

8. Using a Database:

Before performing any operations on a database, you need to select it using the USE statement:

```
USE IMS;
```

9. Creating the tables for each entity:

```
CREATE TABLE Supplier_Details
(Supplier_Id INT(10) PRIMARY KEY,
Supplier_Name VARCHAR(100),
Supplier_Address VARCHAR(100),
Supplier_MobileNumber BIGINT(10),
UNIQUE(Supplier_Name),
UNIQUE(Supplier_MobileNumber));

CREATE TABLE Products_Details
(Product_Id(10) INT PRIMARY KEY,
Product_Name VARCHAR(100),
Product_Description VARCHAR(100),
Product_Category VARCHAR(100),
Product_Quantity BIGINT(50),
Supplier_Name VARCHAR(100),
UNIQUE(Product_Name),
FOREIGN KEY (Supplier_Name) REFERENCES Supplier_Details (Supplier_Name));

CREATE TABLE Customer_Details
(Customer_Id INT(10) PRIMARY KEY,
Customer_Name VARCHAR(100),
Customer_Address VARCHAR(200),
Customer_MobileNumber BIGINT(10),
UNIQUE(Customer_Name),
UNIQUE(Supplier_Address),
UNIQUE(Supplier_MobileNumber));

CREATE TABLE Order_Details
(Order_Id INT(10) PRIMARY KEY,
Order_Quantity BIGINT(50),
Order_Date DATE,
Product_Name VARCHAR(100),
Customer_Name VARCHAR(100),
Customer_Address VARCHAR(200),
Customer_MobileNumber BIGINT(10),
```

```
UNIQUE(Order_Date),
FOREIGN KEY (Product_Name) REFERENCES Product_Details(Product_Name),
FOREIGN KEY (Customer_Name) REFERENCES Customer_Details (Customer_Name),
FOREIGN KEY (Customer_Address) REFERENCES Customer_Details (Customer_Address),
FOREIGN KEY (Customer_MobileNumber) REFERENCES Customer_Details
(Customer_MobileNumber));

CREATE TABLE Delivery_Details
(Delivery_Id INT(10) PRIMARY KEY,
Delivery_Quantity BIGINT(50),
Delivery_Date DATE,
Product_Name VARCHAR(100),
Order_Date DATE,
FOREIGN KEY (Product_Name) REFERENCES Product_Details(Product_Name),
FOREIGN KEY (Order_Date) REFERENCES Order_Details(Order_Date));

CREATE TABLE Defective_Products
(Defective_Quantity BIGINT(50),
Product_Name VARCHAR(100),
Supplier_Name VARCHAR(100),
Supplier_MobileNumber BIGINT(10),
FOREIGN KEY (Product_Name) REFERENCES Product_Details(Product_Name),
FOREIGN KEY (Supplier_Name) REFERENCES Supplier_Details (Supplier_Name),
FOREIGN KEY (Supplier_MobileNumber) REFERENCES Supplier_Details
(Supplier_MobileNumber));
```

10. Insert records:

Add data to your tables to work with. This step helps you test your database.

```
INSERT INTO Supplier_Details
(Supplier_Id, Supplier_Name, Supplier_Address, Supplier_MobileNumber)
VALUES
('S01', 'Kartik_Pawar', 'Babhaleshwar', '7840975898'),
('S02', 'Ishwar Mhase', 'Rahuri', '7746894589');

INSERT INTO Product_Details
(Product_Id, Product_Name, Product_Description, Product_Category, Product_Quantity,
Supplier_Name)
VALUES
('P01', 'IQOO Z6 5G', 'Snapdragon processor', 'Mobile', '20', 'Kartik_Pawar'),
('P02', 'Thomson Android Smart TV', 'Android 11 5000+ Apps Support', 'Appliances', '00',
'Ishwar Mhase');
```

```
INSERT INTO Customer_Details
(Customer_Id, Customer_Name, Customer_Address, Customer_MobileNumber)
VALUES
('C01', 'Bipin_Pawar', 'Shrirampur', '8446435328'),
('C02', 'Sujit Shete', 'Shirdi', '8974943569');

INSERT INTO Order_Details
(Order_Id, Order_Quantity, Order_Date, Product_Name, Customer_Name,
Customer_Address, Customer_MobileNumber)
VALUES
('O01', '2', '05/12/2023', 'IQOO Z6 5G', 'Bipin_Pawar', 'Shrirampur', '8446435328'),
('O02', '1', '05/12/2023', 'Thomson Android Smart TV', 'Ishwar Mhase', 'Sujit Shete',
'Shirdi', '8974943569');

INSERT INTO Delivery_Details
(Delivery_Id, Delivery_Quantity, Delivery_Date, Product_Name, Order_Date)
VALUES
('D01', '2', '09/12/2023', 'IQOO Z6 5G', '05/12/2023'),
('D02', '1', '11/12/2023', 'Thomson Android Smart TV', '05/12/2023');

INSERT INTO Defective_Products
(Defective_Quantity, Product_Name, Supplier_Name, Supplier_MobileNumber)
VALUES
('D01', '1', 'IQOO Z6 5G', 'Kartik_Pawar', '7840975898');
```

11. Select records:

Write SQL queries to retrieve and manage data.

Retrieve all customer:

```
Select * FROM Customer_Details;
```

Retrieve a specific product:

```
Select * from Product_Details where Product_Name=' IQOO Z6 5G';
Select Product_Quantity from Product_Details where Product_Name=' IQOO Z6 5G';
```

Now try similar Select queries with other tables

12. Update records:

Write SQL statements to update record(s) when needed. For example:

Update supplier mobile number:

Update Supplier_Details SET Supplier_MobileNumber = '7868456795' Where Supplier_Name = 'Kartik_Pawar';

13. Delete records:

Write SQL statements to delete record(s) when needed.

Delete FROM Product_Details Where Product_Name = ‘Thomson Android Smart TV’;

PN: Ideally no data should be deleted from any tables. You can use an additional column to set the status of that record to ‘Active/Inactive’, etc. Or you can use an Archive table to move the unnecessary records out of the main table.